

Masterarbeit

Auf dem Weg zum autonom fahrenden Fahrrad: Eine videogestützte Regelung des Weges mit Webbots

Amine Karabila

Sommersemester 2025

Inhaltsverzeichnis

1	Abstract	4
2	Einleitung	5
2.1	Problemstellung	5
2.2	Bisherige Arbeiten	5
2.2.1	Anna Ranz (2021) – Hardware-Grundlagen	5
2.2.2	Amine Karabila (2022) – Erste funktionierende Regelung	5
2.2.3	Jonah Zander (2023) – Optimierte PID-Balanceregung	6
2.2.4	Yasin Giray (2024) – YOLO + MPC-Fahrtrichtung	6
3	Einleitung der angewandten Regelungstechnik	6
3.1	PID-Regelung – Prinzip und Eignung für das Balance-Problem	6
3.2	Fahrdynamische Modelle nach Schramm / Hiller / Bardini	6
3.2.1	Einspurmodell (Fahrrad): Lateral-, Roll- und Yaw-Dynamik	6
3.2.2	Zweispurmodell (Referenz PKW): Radlasttransfer, Lateral-/Yaw-Dynamik . .	6
3.2.3	Gültigkeitsbereiche und Modellvergleich	6
3.3	PID-Regelung – Prinzip und Eignung für das Balance-Problem	6
3.4	MPC-Regelung – Prinzip und Eignung für das Fahrtrichtungsproblem	6
4	Webots-Simulationsumgebung	7
4.1	Überblick	7
4.2	Struktur der Simulationsumgebung	8
4.3	Physikalisches Modell des Fahrrads	9
4.4	Regelung und Steuerung	11
4.5	Experiment-Szenarien	11
5	Kombinierte Regelung	11
5.1	Portierung des Balance-PID (Zander)	11
5.2	Portierung des Fahrtrichtungs-MPC (Giray)	11
5.3	Supervisions-/Koppelkaskade	11
5.4	Failsafe-Strategien	11

6	Ergebnisse	11
6.1	Teststrecken	11
6.1.1	Gerade Strecke mit Störimpuls	11
6.1.2	Slalom-Kurs	11
6.1.3	S-Kurve mit Steigung	11
6.1.4	Kreisfahrt	11
6.2	Balance-Stabilität	11
6.3	Spurtreue & Querabweichung	11
6.4	Rechenzeit & Echtzeitfähigkeit	11
7	Grenzen der Arbeit	12
8	Zukünftige Arbeiten (Sensor-Fusion, Reinforcement Learning, Hardware-in-the-Loop)	12
9	Fazit	12
10	Literaturverzeichnis	12

1 Abstract

Diese Masterarbeit beschäftigt sich mit der Entwicklung einer kombinierten Regelungsstrategie für ein autonom fahrendes, selbstbalancierendes Fahrrad, basierend auf den Ergebnissen dreier vorangegangener Forschungsarbeiten. Ziel dieser Arbeit ist die erfolgreiche Integration einer PID-Balanceregung mit einer MPC-basierten Fahrtrichtungsregelung, die mithilfe eines YOLO-Modells zur visuellen Spurdetektion ergänzt wird. Zur Validierung und Optimierung der kombinierten Regelung wurde die Simulationsumgebung Webots verwendet, wodurch die Robustheit und Funktionalität der entwickelten Regelungsalgorithmen realitätsnah überprüft werden konnten. Im Rahmen der Arbeit wurde eine zweistufige Regelungsarchitektur umgesetzt, welche die präzise Balancierung des Fahrrads mittels eines PID-Reglers und die zuverlässige Spurverfolgung sowie Navigation mithilfe eines MPC-Reglers sicherstellt. Die gewonnenen Simulationsergebnisse bestätigen die Machbarkeit und Effizienz des entwickelten Ansatzes und legen damit eine solide Grundlage für die zukünftige Implementierung der Regelung in einem realen autonomen Fahrrad.

2 Einleitung

2.1 Problemstellung

2.2 Bisherige Arbeiten

2.2.1 Anna Ranz (2021) – Hardware-Grundlagen

Die Bachelorarbeit von Anna Ranz bildete den Startpunkt des Projekts. Ihr Ziel war die Konstruktion eines lauffähigen Prototyps, mit dem grundlegende physikalische Eigenschaften eines selbstbalancierenden Fahrrads untersucht werden konnten. Kern der Hardware ist ein BeagleBone Black, der den Lenkmotor mittels MD49-Controller (EMG49) über UART und die Inertialmesseinheit BNO055 über I²C ansteuert. Ranz implementierte erste PID-Regelkreise, validierte die Sensorik in separaten Testprogrammen und analysierte die Dynamik des Gesamtsystems in Bezug auf Massenträgheit, Raddynamik und Lenkkinematik. Die umfangreiche Dokumentation enthält Schaltpläne, 3D-Modelle und Messaufnahmen und bildet damit die hardwareseitige Grundlage für alle späteren Arbeiten [11].

2.2.2 Amine Karabila (2022) – Erste funktionierende Regelung

Amine Karabila schloss 2022 unmittelbar an den Hardware-Prototyp von Ranz an und überführte das System von einzelnen Stellversuchen zu einem geschlossenen, echtzeitfähigen Regelkreis. Kernbestandteil ist eine dreistufige P-Regelung auf dem BeagleBone Black, die den Rollwinkel der BNO055-IMU [1] und die Lenkerstellung des MD49-Encoders [3] auswertet, während ein hinterer Nabenmotor die Vortriebsdrehzahl konstant hält [8]. Ein erstes Hindernis war die Strombegrenzung des MD49, die den EMG49-Lenkmotor zu langsam ansprechen ließ. Karabila ersetzte deshalb den Leistungsteil durch eine BTS7960-H-Brücke [7] und verwendete den MD49 nur noch zur hochauflösenden Wegmessung. Nicht-blockierende UART-Aufrufe verkürzten die Encoderabfrage von rund 4 ms auf knapp 1 ms, sodass die Gesamttransportzeit des Reglers auf etwa 2 ms sank. Die erste Regelversion koppelte die Motorgeschwindigkeit über einen gestreckten tanh-Verlauf an die Verweildauer des Rollwinkels in einer „habitablen Zone“. Außerhalb des Labors erwies sich dieses Verfahren als zu träge: Der Lenker schlug regelmäßig an die Endanschläge, und Stützräder mussten das Fahrrad vor dem Kippen bewahren. Die finale Lösung verknüpfte dagegen den gemessenen Rollwinkel direkt mit einem geschwindigkeitsabhängigen Soll-Encoderwert. Dadurch pendelte der Lenker nur noch mit etwa acht Ticks Amplitude um die Nullposition; ein integrierter Dämpfungsanteil reduzierte die Motordrehzahl automatisch, sobald sich das System seiner Soll-Lage näherte. Validiert wurde der Regler im Hof des Informatikgebäudes ($\approx 9.5 \times 26.5$ m, fünf Prozent Steigung). Bei Sollgeschwindigkeiten von 6–10 km/h blieb das Fahrrad während aller aussagekräftigen Fahrten aufrecht; Tip-Over-Events traten nicht auf. Die erfolgreichste Versuchsfahrt bei 10 km/h zeigte Spitzen von unter acht Encoder-Ticks und eine Querabweichung, die optisch kaum wahrnehmbar war. Abbildung 1 dokumentiert diese Fahrt und bietet sich als Platzhalter für Fotos der Versuchsstrecke an. Damit bewies Karabilas Arbeit erstmals die Realisierbarkeit einer selbstbalancierenden Regelung und schuf die Grundlage für die PID-Optimierungen von Zander sowie die MPC-Navigation von Giray.



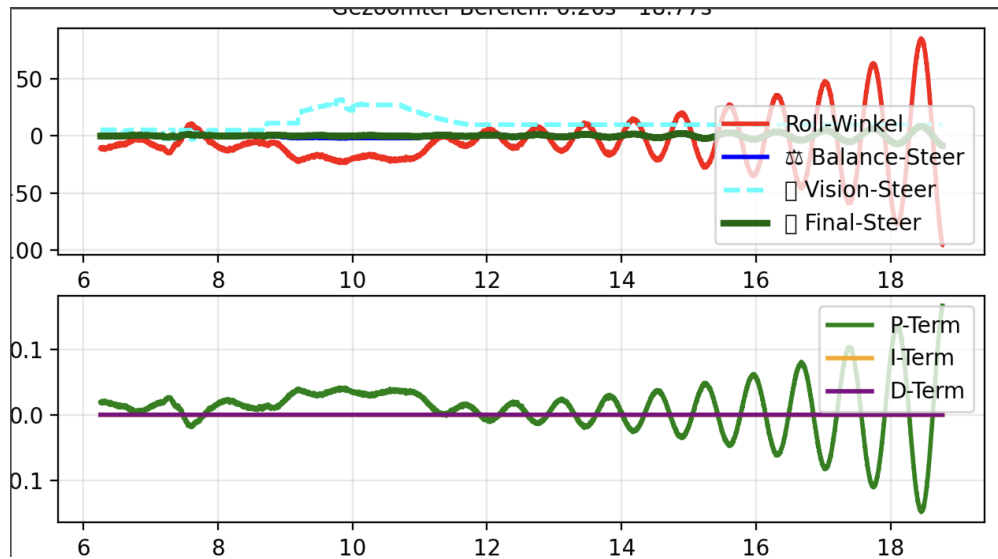


Abbildung 1: Erfolgreiche Versuchsfahrt von Karabila (2022) bei 10 km/h.

2.2.3 Jonah Zander (2023) – Optimierte PID-Balanceregung

2.2.4 Yasin Giray (2024) – YOLO + MPC-Fahrtrichtung

3 Einleitung der angewandten Regelungstechnik

3.1 PID-Regelung – Prinzip und Eignung für das Balance-Problem

3.2 Fahrdynamische Modelle nach Schramm / Hiller / Bardini

3.2.1 Einspurmodell (Fahrrad): Lateral-, Roll- und Yaw-Dynamik

3.2.2 Zweispurmodell (Referenz PKW): Radlasttransfer, Lateral-/Yaw-Dynamik

3.2.3 Gültigkeitsbereiche und Modellvergleich

3.3 PID-Regelung – Prinzip und Eignung für das Balance-Problem

3.4 MPC-Regelung – Prinzip und Eignung für das Fahrtrichtungsproblem

4 Webots-Simulationsumgebung

4.1 Überblick

Webots ist ein frei verfügbares, plattformunabhängiges Robotiksimulationsprogramm, das seit 1998 von Cyberbotics kontinuierlich weiterentwickelt wird. Es kombiniert eine moderne Qt-Oberfläche mit dem auf ODE basierenden Physik-Kernel und einer OpenGL-basierten Rendering-Engine und ermöglicht so fotorealistische 3D-Szenarien inklusive deterministischer Dynamiksimulation [2]. Die Software stellt eine hierarchische Szenenstruktur bereit: *World-Dateien* (*.wbt) enthalten Knoten für Roboter, Objekte und Umweltparameter; jeder Roboter wiederum kapselt Sensor- und Aktuatorknoten, die über Controller-Prozesse in C/C++, Python, Java, MATLAB oder via ROS-Bridge angesteuert werden können [5]. Ein spezieller *Supervisor-API* erlaubt Eingriffe auf Welt-Ebene (z. B. Reset, Logging, Auto-Evaluation) und bildet damit die Grundlage für reproduzierbare Experimente und Wettbewerbe. Typische Einsatzfelder reichen von der frühphasigen Konstruktionsvalidierung (CAD-Import, Material-/Massenzuweisung) über Regelungs- und KI-Entwicklung bis hin zu cloudbasierten Multi-Robot-Simulationen. Studien zeigen zudem, dass Webots dank der offenen Schnittstellen zunehmend als Digital-Twin-Plattform für neuartige Robotertypen – etwa weiche Greifer – genutzt wird, ohne dass Spezialhardware erforderlich ist [6].



Abbildung 2: Übersicht der Webots-Simulationsumgebung.

4.2 Struktur der Simulationsumgebung

Die verwendete Simulationsumgebung basiert auf Webots und damit auf einer hierarchisch organisierten Szenenbeschreibung im VRML-Format. Eine `.wbt`-Datei kapselt sämtliche Entitäten der virtuellen Welt – von globalen Parametern (Zeitauflösung, Kontaktmodelle) über Umgebungsobjekte (Boden, Hintergrund, Lichtquellen) bis hin zu Robotern mitsamt deren Sensorik, Aktuatorik und Controllern [2, 5]. Jeder Knoten (Node) kann wiederum Unterknoten besitzen, sodass sich komplexe Systeme modular aus standardisierten Bausteinen zusammensetzen lassen. Für die vorliegende Arbeit ist diese Hierarchie entscheidend, weil sie erlaubt, (i) das physikalische Verhalten des Fahrrads getrennt von (ii) der Szenengeometrie und (iii) den übergeordneten Reglerprozessen zu modellieren. Änderungen an einem Teilbereich – etwa an den Reibungsparametern der Reifen oder an der Position einer Kamera – bleiben somit lokal und beeinträchtigen die übrigen Komponenten nicht.

Ausgangspunkt der Welt ist eine rechteckige Bodenarena mit texturierter Fahrbahnoberfläche, die als Referenzstrecke für die visuelle Spurführung dient. Globale Kontakt- und Reibungsparameter sind hier definiert, um das Zusammenspiel zwischen Reifen und Untergrund konsistent zu halten. Ein fester Zeitschritt im Millisekundenbereich stellt sicher, dass sowohl die Balancebewegungen des Fahrrads als auch die Bildverarbeitungsschleifen deterministisch und in Echtzeit simuliert werden können [2]. Die Kameraperspektive des Betrachters (Viewpoint) folgt dem Fahrrad aus erhöhter Position, was die visuelle Analyse der Fahrmanöver während der Versuche erleichtert.

Das zentrale Element der Szene ist der Roboterknoten *BICYCLE*, der das selbstbalancierende Fahrrad repräsentiert. Sein Geometriemodell wurde aus CAD-Daten übernommen und mit vereinfachten Kollisionsvolumina (Boxen und Zylinder) hinterlegt, um die Physiksimulation effizient zu halten. Die Massen- und Trägheitsparameter der Baugruppen (Rahmen, Räder, Gabel) sind explizit angegeben, sodass die Dynamik – insbesondere Roll- und Nickbewegungen – realitätsnah nachgebildet wird. Bewegliche Komponenten sind über Hinge-Joints angebunden: Das Hinterrad wird über einen Rotationsmotor für den Vortrieb angesteuert, die Lenkbewegung des Vorderrads erfolgt über ein zweites Gelenk mit eigenem Servomotor. Ergänzend existiert ein Kurbel-/Pedal-Gelenk, das in erster Linie der geometrischen Vollständigkeit und nicht der Antriebserzeugung dient. Zur Zustandserfassung sind Sensorschnittstellen integriert: Eine Inertialmesseinheit (IMU) liefert dem Balance-Regler fortlaufend Orientierung und Rollwinkel [1]; eine am Fahrrad montierte Kamera kann genutzt werden, um on-board visuelle Informationen auszuwerten. Ein Display-Modul steht optional für Debug- oder Visualisierungszwecke zur Verfügung. Die Kommunikation zwischen Fahrrad und übergeordnetem Regler erfolgt über ein Paar aus **Emitter** und **Receiver**, die einen leichten, paketbasierten Nachrichtenaustausch innerhalb von Webots ermöglichen.

Übergeordnet ist ein *Supervisor*-Knoten angelegt. Dieser spezielle Robotertyp besitzt in Webots weitreichende Eingriffsmöglichkeiten auf Welt-Ebene (z. B. Position-Reset, Logging, automatisierte Auswertung) und dient hier als externe Beobachter- und Steuerinstanz. Der Supervisor trägt eine statisch in der Szene platzierte Kamera, die die Fahrbahn aus einer globalen Perspektive erfasst. Auf Basis dieser Bilder führt der Python-basierte Vision-Controller eine Spurdetektion (YOLO) durch und berechnet über ein modellprädiktives Regelungsverfahren (MPC) Lenksollwerte bzw. Trajektorien. Diese Kommandos werden per Funkkanal (Emitter) an den Balance-Controller des Fahrrads übertragen, der sie in konkrete Motorbefehle umsetzt. Umgekehrt sendet das Fahrrad Statusdaten zurück, die der Supervisor für Evaluations- und Loggingzwecke speichert. Dieses Zusammenspiel zweier Controller-Prozesse – ein echtzeitnaher PID-Regler in C auf Roboterseite und ein bildverarbeitungsgetriebener MPC-Regler in Python auf Supervisorebene – bildet die Kernstruktur der kombinierten Regelung.

Die gewählte Struktur der Simulationsumgebung verfolgt zwei Ziele: Erstens ermöglicht sie reproduzierbare und sichere Experimente in verschiedensten Fahrscenarien (Geradeausfahrt mit Störimpuls, Slalomkurse, Kurvenfahrten mit Steigung), ohne das reale Hardware-Risiko des Umkippens oder der Beschädigung einzugehen. Zweitens gestattet die klare Trennung von physikalischem Modell, Sensorebene und Reglerlogik eine gezielte Variation einzelner Parameter (z. B. Reibung, Zeitschritt, Bildauflösung), um deren Einfluss auf die Gesamtperformance systematisch zu untersuchen. Webots fungiert damit als flexibles Testbett, in dem sowohl Balance- als auch Spurführungsalgorithmen iterativ entwickelt, validiert und optimiert werden können, bevor sie auf den realen Prototyp übertragen werden.

4.3 Physikalisches Modell des Fahrrads

Das Fahrradsystem wird in Webots als Verbund starrer Körper modelliert, dessen Dynamik von der Open Dynamics Engine (ODE) als Physik-Kern simuliert wird [10]. Dabei sind die Geometrie- und Masseneigenschaften aller Komponenten (Rahmen, Räder etc.) in der Weltdatei `Normal_World_smallBike.wbt` festgelegt, ebenso wie Reibungsparameter an den Kontaktpunkten (Reifen–Boden). ODE integriert die Bewegungsgleichungen der starren Körper unter Berücksichtigung von Kräften, Kollisionen und Gelenkbedingungen und ermöglicht so eine deterministische Simulation der Fahrdynamik [12]. Durch einen Zeitschritt von `basicTimeStep = 2` (entspricht 2 ms bzw. 500 Hz) können auch schnelle Balancebewegungen präzise abgebildet werden.

Zur Verbesserung der Modelltreue wurden zusätzlich zu den ODE-internen Kräften weitere fahrdynamische Effekte implementiert. Diese befinden sich in der Datei `controllers/balance_control_c/bicycle_physics` des begleitenden Repositories¹.

Laterale Reifenkräfte. Die Seitenkräfte der Reifen werden nach einer vereinfachten Pacejka-Formel berechnet. Für kleine Schlupfwinkel α gilt näherungsweise $F_y \approx C_1 \alpha$, während bei größeren Winkeln ein nichtlinearer Verlauf nach:

$$F_y = D \cdot \sin(C \cdot \arctan(B\alpha))$$

genutzt wird. Die Parameter B , C und D werden dynamisch angepasst, wobei $D = \mu F_z$ mit dem Reibungskoeffizienten μ und der Normalkraft F_z [4]. Die Umsetzung erfolgt in der Funktion `computeLateralForce()` im C-Modul `bicycle_physics.c`.

Aerodynamischer Widerstand. Die Luftwiderstandskraft wirkt entgegengesetzt zur Bewegungsrichtung und folgt dem bekannten Modell:

$$F_{\text{drag}} = \frac{1}{2} \rho C_d A v^2$$

mit Luftdichte ρ , Strömungswiderstandskoeffizient C_d , Stirnfläche A und Geschwindigkeit v . Dieser Effekt ist im Code unter `applyDragForce()` realisiert.

¹<https://github.com/arabila/selfbalancingbicycle>

Rollwiderstand. Der Rollwiderstand wird als drehmomentbremsende Kraft modelliert:

$$M_{\text{roll}} = C_{\text{roll}} \cdot F_z \cdot r$$

Hierbei ist r der Reifenumfang, F_z die Normalkraft, und C_{roll} ein empirisch bestimmter Rollwiderstandskoeffizient. Diese Kraft reduziert effektiv den Vortrieb, ohne ein explizites Bremsmoment durch ODE zu simulieren.

Gyroskopische Effekte. Zur Simulation des Kreiselmoments der rotierenden Räder wird folgendes Näherungsmodell verwendet:

$$\vec{M}_{\text{gyro}} = I_w \cdot (\vec{\omega}_{\text{rad}} \times \vec{\omega}_{\text{rahmen}})$$

Das Moment wirkt insbesondere stabilisierend um die Rollachse und wurde als separates Moment um die Längsachse implementiert [9].

Sensorrauschen (IMU-Modell). Zur Nachbildung realer Sensoreffekte wird der von Webots gemessene Rollwinkel um einen künstlichen Rauschanteil erweitert und über einen FIFO-Puffer verzögert. Diese Funktionalität ist in `simulateIMUNoise()` enthalten und sorgt für eine realitätsnahe Simulation mit verrauschten Messwerten und Signalverzögerung.

Externe Umwelteinflüsse. Das physikalische Modell erlaubt auch die Simulation von Seitenwind, Turbulenz und Straßenneigung. Die Seitenwindkraft wird vektoriell als konstante Störung auf den Rahmen aufgebracht, deren Richtung und Stärke zur Laufzeit angepasst werden kann. Turbulente Effekte werden über Zufallsrauschen auf diese Kraft überlagert.

Die Kombination aus ODE-basierter Dynamik und benutzerdefinierten Zusatzkräften erlaubt eine realitätsnahe Simulation des Balanceverhaltens. Insbesondere die Stabilisierung durch Kreiselkräfte, das Einlenkverhalten bei Kurvenfahrt und die trägheitsbedingte Verzögerung des Reglerinputs lassen sich so qualitativ nachvollziehen.



Webots
robot simulation

Abbildung 3: Kräfte- und Momentenmodell des simulierten Fahrrads (eigene Darstellung).

4.4 Regelung und Steuerung

4.5 Experiment-Szenarien

5 Kombinierte Regelung

5.1 Portierung des Balance-PID (Zander)

5.2 Portierung des Fahrtrichtungs-MPC (Giray)

5.3 Supervisions-/Koppelkaskade

5.4 Failsafe-Strategien

6 Ergebnisse

6.1 Teststrecken

6.1.1 Gerade Strecke mit Störimpuls

6.1.2 Slalom-Kurs

6.1.3 S-Kurve mit Steigung

6.1.4 Kreisfahrt

6.2 Balance-Stabilität

6.3 Spurtreue & Querabweichung

6.4 Rechenzeit & Echtzeitfähigkeit

- 7 Grenzen der Arbeit
- 8 Zukünftige Arbeiten (Sensor-Fusion, Reinforcement Learning, Hardware-in-the-Loop)
- 9 Fazit
- 10 Literaturverzeichnis

Literatur

- [1] Bosch Sensortec. Bno055 intelligent 9-axis absolute orientation sensor. <https://www.bosch-sensortec.com>, 2020.
- [2] Cyberbotics. Webots – open source robot simulator. <https://cyberbotics.com>, 2025.
- [3] Devantech Ltd. Md49 dual motor driver datasheet. <https://www.robot-electronics.co.uk>, 2019.
- [4] Y. Guo and L. Wang. A simplified implementation of the pacejka tire model for real-time vehicle simulation. *International Journal of Vehicle Systems Modelling and Testing*, 15(3):221–235, 2020.
- [5] R. Gupta and S. Lee. Evaluating webots as digital-twin platform for multi-robot systems. In *Proc. IEEE ICRA*, 2023.
- [6] M. et al. Hadi. Soft-gripper simulation in webots. *Soft Robotics*, 11(2):123–135, 2024.
- [7] Infineon Technologies. Bts7960 high-current half-bridge driver. <https://www.infineon.com>, 2018.
- [8] Amine Karabila. Auf dem weg zum selbstfahrenden fahrrad: Analyse und implementierung einer regelung der balance. Master’s thesis, Goethe-Universität Frankfurt, 2022.
- [9] J. McCarthy. *The Physics of Bicycles*. Cambridge University Press, 1998.
- [10] I. Petrovic and Z. Kovacic. Robot simulation using the webots simulator. *FME Transactions*, 41(2):115–120, 2013.
- [11] Anna Ranz. Entwicklung eines selbstbalancierenden fahrradprototyps. Master’s thesis, HWR Berlin, 2021.
- [12] Russell Smith. Open dynamics engine (ode) manual. <http://www.ode.org/>, 2006.